TP3 : Modularisation et organisation de projet

Enseignant : Abderrahmane Kasmi | **Public :** 1ère année Cycle Ingénieur ISEN

1 Introduction

Ce TP a pour objectif de vous faire construire un petit projet Python organisé en modules et en packages, en respectant l'architecture standard d'un projet et en écrivant correctement tous les imports nécessaires.

Objectifs du TP

À l'issue de ce TP, vous serez capable de :

- Mettre en place l'arborescence standard d'un projet Python (`src`, `README.md`).
- Découper un programme en modules et en packages.
- Utiliser les imports globaux, ciblés et avec alias.
- Choisir entre imports absolus et imports relatifs.
- Exploiter les fichiers `__init__.py` pour simplifier les imports.
- Protéger le point d'entrée d'un programme avec `if __name__ == "__main__"`.

2 Exercice — Mini-projet `convertisseur`

2.1 Description du projet

Vous devez développer `convertisseur`, une petite bibliothèque de conversion d'unités avec un programme de démonstration. Elle permet de convertir :

- des **températures** : Celsius → Fahrenheit et Celsius → Kelvin ;
- des **longueurs** : kilomètres → miles et miles → kilomètres.

Un module d'affichage se charge de mettre en forme les résultats, et un fichier de constantes regroupe les valeurs utilisées dans les calculs.

2.2 Architecture à créer

Créez le projet `tp3_convertisseur/` avec l'arborescence suivante :

```
tp3_convertisseur/
|-- README.md
|-- src/
|   `-- convertisseur/
|       |-- __init__.py
|       |-- constantes.py
|       |-- affichage.py
|       |-- main.py
|       `-- conversions/
|           |-- __init__.py
|           |-- temperature.py
|           `-- longueur.py
```

Le fichier à la racine doivent contenir :

— `README.md` : le titre du projet, une courte description et la commande pour le lancer ;

2.3 Contenu des modules et imports à écrire

Le tableau suivant décrit le contenu de chaque fichier Python **et** les imports que vous devez y écrire. Respectez **exactement** le type d'import demandé.

Fichier	Contenu attendu et imports
`constantes.py`	**Contenu :** deux constantes, `KELVIN_OFFSET = 273.15` et `MILES_PAR_KM = 0.621371`. **Imports :** aucun.
`conversions/temperature.py`	**Contenu :** `celsius_vers_fahrenheit(c)` et `celsius_vers_kelvin(c)`. **Imports :** import **absolu** de la constante `KELVIN_OFFSET` depuis `convertisseur.constantes`.
`conversions/longueur.py`	**Contenu :** `km_vers_miles(km)` et `miles_vers_km(miles)`. **Imports :** import **relatif vers le package parent** de la constante `MILES_PAR_KM`.
`conversions/__init__.py`	**Contenu :** exposer les quatre fonctions de conversion. **Imports :** imports **relatifs** depuis les modules `temperature` et `longueur`.
`affichage.py`	**Contenu :** `entete()`, qui renvoie la chaîne `"Convertisseur - AAAA-MM-JJ"` avec la date du jour, et `formater(valeur, unite)`, qui renvoie par exemple `"77.00 F"` (deux décimales). **Imports :** import **avec alias** : `import datetime as dt`.
`__init__.py` (racine)	**Contenu :** la variable `__version__ = "0.1.0"`, et l'exposition à la racine du package des quatre fonctions de conversion ainsi que de `entete` et `formater`. **Imports :** imports **relatifs** depuis `.conversions` et `.affichage`.
`main.py`	**Contenu :** une fonction `main()` qui exécute le scénario décrit plus bas. **Imports :** import **ciblé** (`from convertisseur import ...`) de tout ce dont le fichier a besoin, **uniquement depuis la racine du package**.

Rappels

- Un import relatif commence par un point (`.`) pour le package courant, deux points (`..`) pour le package parent.
- Le fichier `__init__.py` d'un package est exécuté lors de son importation : c'est lui qui permet de simplifier les imports.
- Un fichier Python peut être lancé comme script (`python fichier.py`) ou comme module (`python -m package.module`).

2.4 Scénario d'exécution

Dans la fonction `main()`, affichez d'abord l'en-tête, puis les résultats des conversions suivantes (en utilisant `formater`) :

- $25\,°$C en Fahrenheit et en Kelvin ;
- 10 km en miles ;
- 5 miles en kilomètres.

La fonction `main()` ne doit être appelée que si le fichier est lancé directement (garde `if __name__ == "__main__"`). Lancez l'application depuis le dossier `src/` :

```
python -m convertisseur.main
```

La sortie attendue est la suivante :

```
Convertisseur - 2026-10-01
77.00 F
298.15 K
6.21 miles
8.05 km
```

2.5 Livrables

Déposez une archive `.zip` du dossier `tp3_convertisseur/` contenant le projet complet, avec l'arborescence demandée, et des imports conformes au tableau.